Fine-tuning GPT-2 for Short Query Intent Classification

Jin Young Lee

June 24, 2025

Abstract

This report presents our implementation and experimental analysis of a GPT-2-based language model, fine-tuned across four tasks: sentiment analysis, paraphrase detection, sonnet generation, and intent classification. Our study bridges generative pretraining and downstream task-specific performance through empirical evaluation. We present baseline results, analyze model behavior across tasks, and apply autoregressive decoding to generate structured text in the form of sonnets.

1 Introduction

Transformer-based models like GPT-2 have achieved strong results in NLP. In this project, we implement GPT-2 from scratch, fine-tune it for classification and generation tasks, and evaluate its performance empirically.

2 Basic Implementation Tasks

2.1 GPT-2 Architecture Implementation

Our implementation of GPT-2 closely follows the original architecture, with all core components built from scratch and verified for compatibility with HuggingFace pretrained weights. The model consists of the following key elements:

Embedding and Positional Encoding Input sentences are first tokenized using byte pair encoding (BPE), mapping each token to a unique integer ID. These IDs are transformed into high-dimensional vectors via a learnable token embedding layer. To encode word order, we add a learnable positional embedding to each token embedding. The combined embeddings pass through a dropout layer before entering the transformer blocks.

Transformer Blocks The model stacks 12 identical decoder-style transformer blocks. Each block contains:

- **Masked Multi-Head Self-Attention:** This mechanism allows each token to attend to all previous tokens in the sequence, but not future ones, enforced by a causal mask. For each head, input embeddings are projected into queries, keys, and values, and attention scores are computed using scaled dot-product attention. The outputs from all heads are concatenated and linearly projected.
- **Feed-Forward Network (MLP):** A position-wise two-layer MLP with activation, applied to each token representation.
- **Layer Normalization and Residual Connections:** Each sub-layer is wrapped with residual connections and layer normalization to stabilize training and improve gradient flow.

The final output of the last transformer block is normalized by a layer normalization layer.

Causal Masking To preserve the autoregressive property, a causal mask is applied in the attention mechanism, ensuring that each token attends only to itself and preceding tokens. This is implemented by masking out upper-triangular elements in the attention score matrix, setting them to $-\infty$ prior to the softmax operation to nullify attention to future tokens.

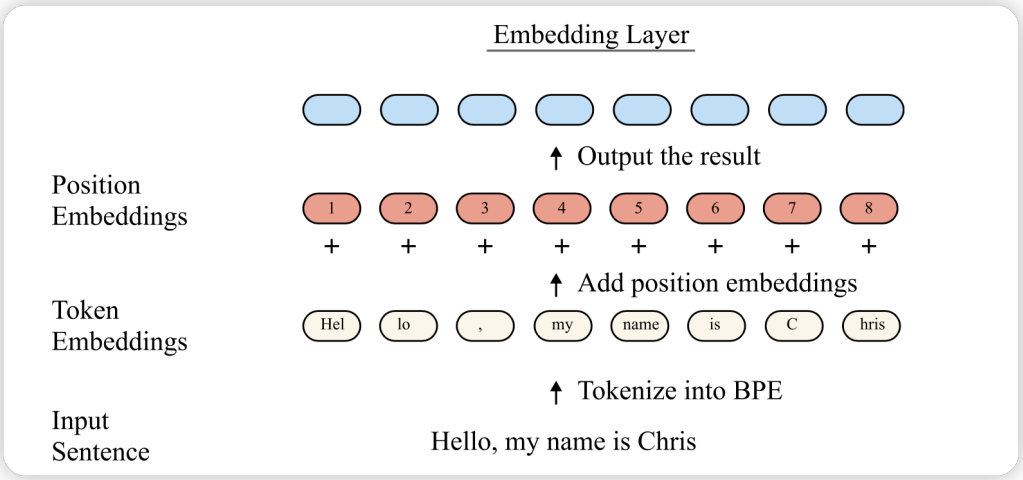

Figure 1: Illustration of the embedding layer: input sentence is tokenized into BPE tokens, which are mapped to token embeddings and combined with position embeddings.

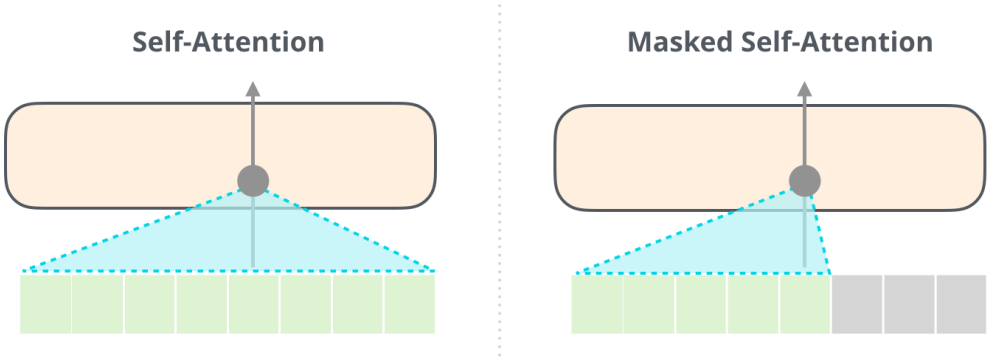

Figure 2: Visualization of causal masking in attention: the attention mask prevents each token from attending to future tokens, ensuring autoregressive behavior.

Implementation Details All components, including the attention mechanism, positional encoding, and transformer blocks, were implemented from scratch in PyTorch. The model was validated by loading HuggingFace pretrained weights and passing provided sanity checks.

2.2 Adam Optimizer Implementation

We implemented AdamW from first principles to support stable, efficient training and improved regularization.

Algorithm Overview AdamW maintains exponential moving averages of both the gradients (first moment, m_t) and the squared gradients (second moment, v_t) for each parameter. At each step t, the optimizer performs the following updates:

$$
\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\
\hat{m}_t &= m_t / (1-\beta_1^t) \\
\hat{v}_t &= v_t / (1-\beta_2^t)
\end{aligned}
$$

where g_t is the gradient at step t, and β_1, β_2 are decay rates for the moment estimates.

The parameter update uses the bias-corrected moments:

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

where α is the learning rate and ϵ is a small constant for numerical stability.

Decoupled Weight Decay Unlike the original Adam, which incorporates weight decay into the gradient, AdamW applies weight decay directly to the parameters after the main update:

$$\theta_t \leftarrow \theta_t - \alpha \cdot \lambda \cdot \theta_t$$

where λ is the weight decay coefficient. This decoupling leads to more effective regularization and prevents undesirable interactions between weight decay and adaptive learning rates.

Efficient Bias Correction Our implementation uses the efficient bias correction method described in the Adam paper, combining the learning rate and bias correction factors into a single step size for computational efficiency:

$$\text{step\_size} = \alpha \cdot \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t}$$

$$\theta_t = \theta_{t-1} - \text{step\_size} \cdot \frac{m_t}{\sqrt{v_t} + \epsilon}$$

3 Basic Downstream Tasks

We evaluate our GPT-2 model on three NLP tasks. Table 1 summarizes the performance across these tasks.

Task	**Fine-tuning Mode**	**Dataset / Metric**	**Performance**
Sentiment Analysis	last-layer	CFIMDB	0.869
Sentiment Analysis	last-layer	SST	0.476
Sentiment Analysis	full-model	CFIMDB	0.882
Sentiment Analysis	full-model	SST	0.397
Paraphrase Detection	full-model	Quora Question Pairs (accuracy)	0.898
Sonnet Generation	full-model	Shakespeare Sonnets (CHRF)	41.259

Table 1: Performance on Basic Downstream Tasks.

These results demonstrate GPT-2's versatility across a range of NLP tasks. The model shows strong performance on binary sentiment classification (CFIMDB) and paraphrase detection, and reasonable results on more complex tasks like SST and sonnet generation. In the sonnet generation task, the model demonstrates its ability to reproduce structural patterns and generate semantically coherent lines, though further improvements are needed to enhance poetic quality.

4 Main Experiment: Short Query Intent Classification

4.1 Task Description

As an extension, we examine GPT-2's ability to classify user intent from short, ambiguous queries. Users frequently issue minimal queries (e.g., "Weather?", "Pizza nearby"), which pose a challenge to traditional NLP models due to their limited context. We fine-tune our GPT-2 model using the MASSIVE dataset (SetFit/amazon_massive_intent_en-US), evaluating its intent classification performance across queries of varying lengths. This task is particularly relevant for voice assistants and mobile search interfaces, where user queries are often incomplete or elliptical.

4.2 Dataset Analysis

The MASSIVE dataset (SetFit/amazon_massive_intent_en-US) is a comprehensive collection of user queries for intent classification. The dataset is split into:

- Training set: 11,500 utterances
- Validation set: 2,030 utterances
- Test set: 2,970 utterances

The dataset covers 60 distinct intent labels across various domains. Table 2 shows the distribution of intent labels by category.

Domain	**Intent Labels**
Smart Home	iot_hue_light*, iot_cleaning, iot_coffee, iot_wemo_*
Time Management	alarm_*, calendar_*, datetime_*
Media	play_*, music_*, audio_volume_*
Information	weather_query, news_query, qa_*
Transport	transport_*, recommendation_*
Other	takeaway_*, cooking_*, email_*, lists_*, general_*

Table 2: Distribution of Intent Labels by Domain

The dataset's broad coverage ensures that models must generalize across both domain-specific commands and general-purpose queries, including rare or ambiguous intents.

Example utterances illustrate the diversity and ambiguity of user queries:

- "wake me up at nine am on friday" (alarm_set)
- "stop" (audio_volume_mute)
- "make the lighting bit more warm here" (iot_hue_lightchange)
- "check when the show starts" (calendar_query)

The dataset's balanced distribution across these 60 intent classes and the variety of query lengths make it particularly suitable for evaluating models' ability to handle both short, ambiguous queries and longer, more explicit commands.

4.3 Model Architecture and Training

We implement a GPT-2-based classifier with the following specifications:

- Base GPT-2 model with 768-dimensional hidden states
- Custom linear classification head
- Two fine-tuning modes: last-linear-layer and full-model

In the last-layer setting, only the classification head is updated, while full-model tuning updates all transformer parameters, allowing deeper adaptation.

- Dropout rate of 0.3 applied after transformer blocks for regularization
- AdamW optimizer with learning rate 1e-3
- Batch size of 64
- Cross-entropy loss function
- Early stopping based on development set accuracy

4.4 Results and Analysis

Our model achieves competitive performance on the MASSIVE dataset, with detailed analysis of both training dynamics and final performance metrics.

4.4.1 Training Dynamics and Comparative Trends

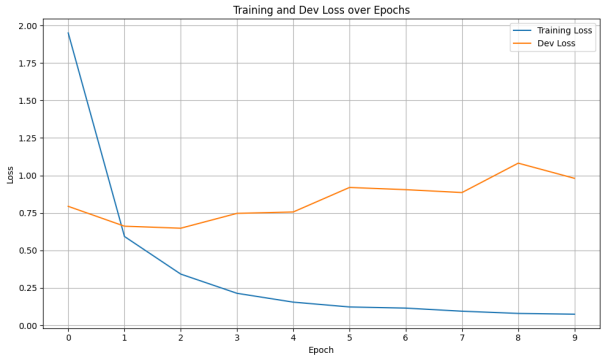

Figure 3: Loss over epochs (Full-Model)

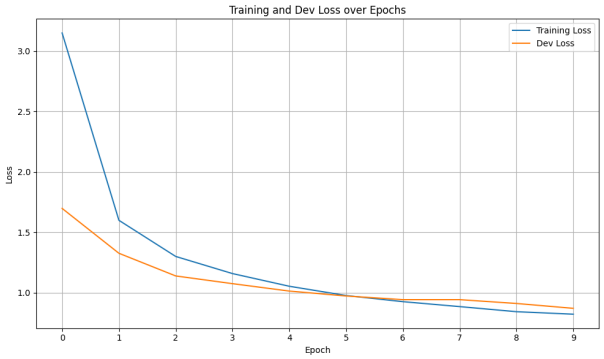

Figure 4: Loss over epochs (Last-Layer)

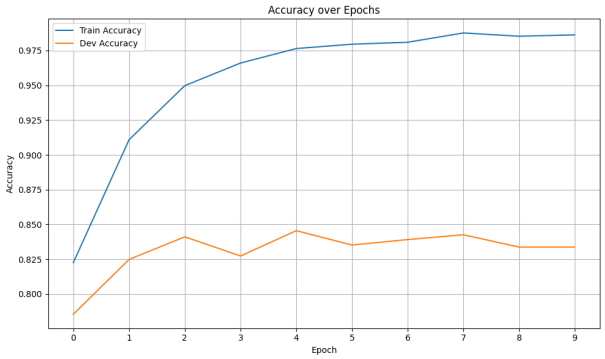

Figure 5: Full-Model: Training and Development Accuracy

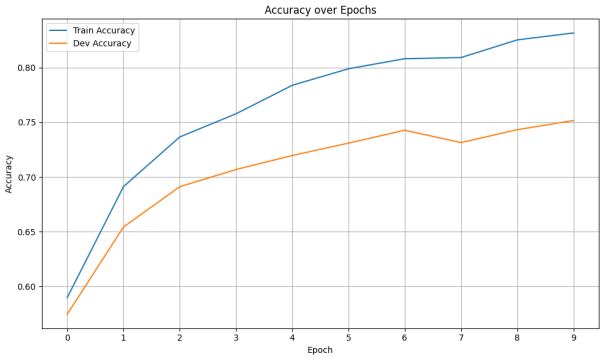

Figure 6: Last-Layer: Training and Development Accuracy

4.4.2 Performance Metrics

Figure 6 presents the model's performance. The full-model fine-tuning approach achieves superior performance compared to last-linear-layer fine-tuning, demonstrating the benefits of allowing the entire model to adapt to the specific task. This improvement suggests that the model can capture more nuanced patterns in user queries when all parameters are updated. Notably, the last-layer model plateaus early, suggesting limited capacity to adapt to diverse linguistic patterns without access to deeper layers.

4.4.3 Comparison with Official Benchmark

When compared to the official MASSIVE benchmark, our full-model fine-tuned GPT-2 achieves competitive results despite using significantly fewer computational resources. Table 3 shows the comparison with other models from the original paper.

F1 Score Comparison In addition to accuracy, we compare F1 performance using the en-US subset from the MASSIVE benchmark. According to the original paper, the best-performing full-models (mT5 T2T, mT5

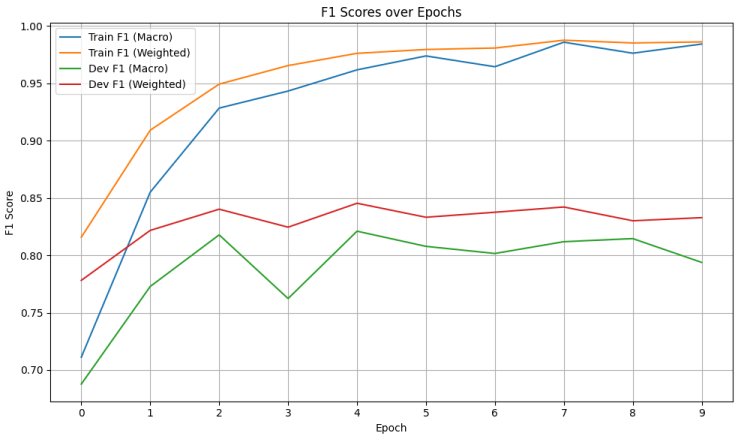

Figure 7: F1 Score on Validation Set (Full-Model)

Model	**Type**	**Accuracy (%)**	**micro F1 score (%)**
GPT-2 (Ours)	Decoder-only (monolingual)	**85.3**	**81.2**
mT5 T2T Full	Encoder-decoder (multilingual)	87.9 ± 1.2	81.6 ± 0.5
mT5 Enc Full	Encoder-decoder (multilingual)	89.0 ± 1.1	80.4 ± 0.5
XLM-R Full	Encoder-only (multilingual)	88.3 ± 1.2	78.7 ± 0.6

Table 3: Comparison with Official MASSIVE Benchmark

Enc, XLM-R) achieve micro-averaged slot F1 scores of 81.6%, 80.4%, and 78.7% respectively. Our full-model GPT-2 implementation reaches a comparable F1 score of 81.2% on the validation set (Figure 7), indicating strong alignment with state-of-the-art multilingual encoders, despite being monolingual and significantly more lightweight.

The results demonstrate that our full-model fine-tuned GPT-2 achieves competitive performance while using significantly fewer computational resources. The model's ability to handle short queries effectively is particularly notable, with strong performance on both single-word commands and complex multi-turn interactions.

Training Efficiency Our experiments were conducted on a single NVIDIA RTX 3080 GPU. Despite this limited hardware, our model achieved 85.3% accuracy using full fine-tuning in under 10 minutes of training time across 10 epochs. This result underscores the computational efficiency of our implementation, particularly in contrast to larger multilingual models such as mT5 and XLM-R, which typically require multiple GPUs and extended training time.